

5 KEYS OF OOP

Let's break down exactly what each one looks like in Dart and how they differ in code.

1. Encapsulation (Protection)

What it looks like: Private variables (`_` in Dart) and public getter/setter methods.

How it's different: It has nothing to do with connecting classes together. It only cares about putting a protective shield around a single object's data so outside code can't corrupt it.

```
class BankAccount {  
    // LOOKS LIKE: Underscore makes it private. The outside world cannot touch  
    // this directly.  
    double _balance = 0;  
  
    // Public way to look at the data (Getter)  
    double get balance => _balance;  
  
    // Public way to change the data safely (Setter)  
    void deposit(double amount) {  
        if (amount > 0) {  
            _balance += amount; // Safe!  
        }  
    }  
}
```

2. Inheritance (Sharing)

What it looks like: The `extends` keyword.

How it's different: It allows a Child class to automatically get all the methods and properties of a Parent class without writing them again. It establishes an "is-a" relationship (A Car *is a* Vehicle).

```
class Vehicle {  
    void startEngine() {  
        print("Engine is running...");  
    }  
}
```

```

}

// LOOKS LIKE: 'extends' means Car gets everything Vehicle has.
class Car extends Vehicle {
    int wheels = 4;
}

void main() {
    Car myCar = Car();
    myCar.startEngine(); // Inherited from Vehicle! We didn't write it in Car.
}

```

3. Polymorphism (Flexibility)

What it looks like: The `@override` annotation, and putting Child objects into Parent variables (Upcasting).

How it's different: It allows you to write one piece of code that can handle many different types of objects, as long as they share a parent. The behavior changes depending on the actual object.

```

class Animal {
    void makeSound() => print("Some generic sound");
}

class Dog extends Animal {
    @override // LOOKS LIKE: Changing the parent's behavior
    void makeSound() => print("Bark!");
}

class Cat extends Animal {
    @override
    void makeSound() => print("Meow!");
}

void main() {
    // LOOKS LIKE: A list of the Parent type holding Child objects.
    List<Animal> myPets = [Dog(), Cat()];

    for (var pet in myPets) {

```

```

    pet.makeSound(); // Polymorphism in action! It outputs "Bark!" then
    "Meow!"
  }
}

```

4. Abstract Class (The Template)

What it looks like: The abstract keyword before class. Methods with no bodies (no {}).

How it's different: It is a class that **cannot be instantiated** (you can't do `Animal()`). It is meant *only* to be a parent. It can have fully working methods, but also "blank" methods that force the child to figure it out.

```

// LOOKS LIKE: 'abstract' keyword
abstract class Shape {
  // A regular, working method
  void printInfo() => print("I am a shape");

  // LOOKS LIKE: A method with no body. The child MUST write this.
  double calculateArea();
}

class Square extends Shape {
  double side;
  Square(this.side);

  @override
  double calculateArea() {
    return side * side; // We are forced to provide the math here
  }
}

```

5. Interface (The Strict Contract)

What it looks like: The implements keyword. (In Dart, we usually use abstract class to define the interface, but the magic happens when we use implements).

How it's different: Unlike Inheritance (extends), where you *share* code, an Interface (implements) gives you **zero code**. It only gives you the *names* of the methods. You must rewrite absolutely everything. Why? Because it guarantees a class has certain abilities, even if it has a totally different parent.

```

abstract class Flyable {
    void fly(); // Just a rule: "You must be able to fly"
}

// LOOKS LIKE: 'implements' keyword
class Bird implements Flyable {
    @override
    void fly() {
        print("Flapping wings!"); // I have to write the code
    }
}

class Airplane implements Flyable {
    @override
    void fly() {
        print("Jet engines roaring!"); // I have to write the code
    }
}

// Both are Flyable, but they share NO code. They just signed the same
contract.

```

Quick Visual Summary: "What they look like"

Concept	The Keyword/Symbol	What you see in code
Encapsulation	_, get, set	String _password;
Inheritance	extends	class Child extends Parent
Polymorphism	@override, Upcasting	Parent obj = Child();
Abstract	abstract	abstract class Name
Interface	implements	class ClassA implements ClassB